

Machine Learning

Qian Wang

School of Biomedical Engineering

ShanghaiTech University

Machine Learning

- An agent is learning if it improves its performance after making observations about the world.
- When the agent is a computer, we call it **machine learning**.
 - A computer observes some data, builds a model based on the data.
 - It uses the model as both a hypothesis about the world and a piece of software that can solve problems.



Types of Learning

- According to the types of feedback that can accompany the inputs, there are three main types of learning.
 - **Supervised learning:**
 - The agent observes input-output pairs and learns a function that maps from input to output.
 - **Unsupervised learning:**
 - The agent learns patterns in the input without any explicit feedback.
 - **Reinforcement learning:**
 - The agent learns from a series of reinforcements: rewards and punishments.

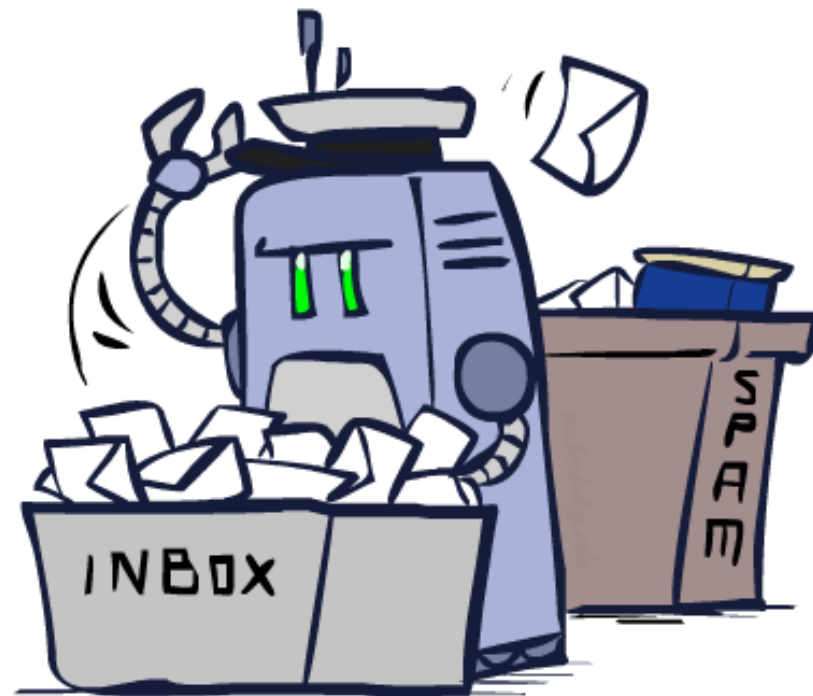
Types of Learning

	<i>Supervised Learning</i>	<i>Unsupervised Learning</i>
<i>Discrete</i>	classification or categorization	clustering
<i>Continuous</i>	regression	dimensionality reduction

Types of Learning

- Classification involves predicting discrete labels or categories for new observations based on a training dataset with known labels.
- Regression is the process of predicting a continuous numeric value for new observations based on a training dataset with known numeric outcomes.
- Clustering is the technique of grouping a set of objects in such a way that objects in the same group (cluster) are more similar to each other than to those in other groups.
- Dimensionality reduction is the process of reducing the number of features under consideration, by obtaining a set of principal features or lower-dimensional representations.

Classification



Example: Spam Filter

- Input: an email.
- Output: spam/ham label.
- Setup:
 - Get a large collection of example emails, each labeled “spam” or “ham”.
 - Note: someone has to hand label all this data!
 - Want to learn to predict labels of new future emails.
- Features: The attributes used to make the ham / spam decision.
 - Words: FREE!
 - Text Patterns: \$dd, CAPS;
 - Non-text: SenderInContacts, WidelyBroadcast;
 - ...



Dear Sir.

First, I must solicit your confidence in this transaction, this is by virtue of its nature as being utterly confidential and top secret. ...



TO BE REMOVED FROM FUTURE MAILINGS,
SIMPLY REPLY TO THIS MESSAGE AND PUT
"REMOVE" IN THE SUBJECT.

99 MILLION EMAIL ADDRESSES
FOR ONLY \$99



Ok, I know this is blatantly OT but I'm beginning to go insane. Had an old Dell Dimension XPS sitting in the corner and decided to put it to use, I know it was working pre being stuck in the corner, but when I plugged it in, hit the power nothing happened.

Example: Digit Recognition

- Input: images / pixel grids.
- Output: a digit 0-9.
- Setup:
 - Get a large collection of example images, each labeled with a digit.
 - Note: someone has to hand label all this data!
 - Want to learn to predict labels of new future digit images.
- Features: The attributes used to make the digit decision
 - Pixels: (6,8)=ON
 - Shape Patterns: NumComponents, AspectRatio, NumLoops
 - ...
 - Features are increasingly induced rather than crafted

 0

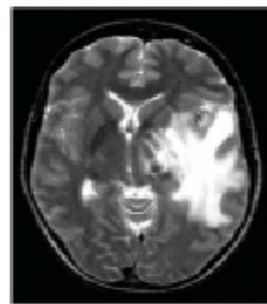
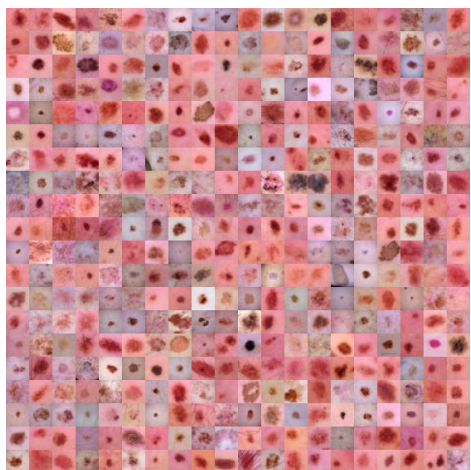
 1

 2

 1

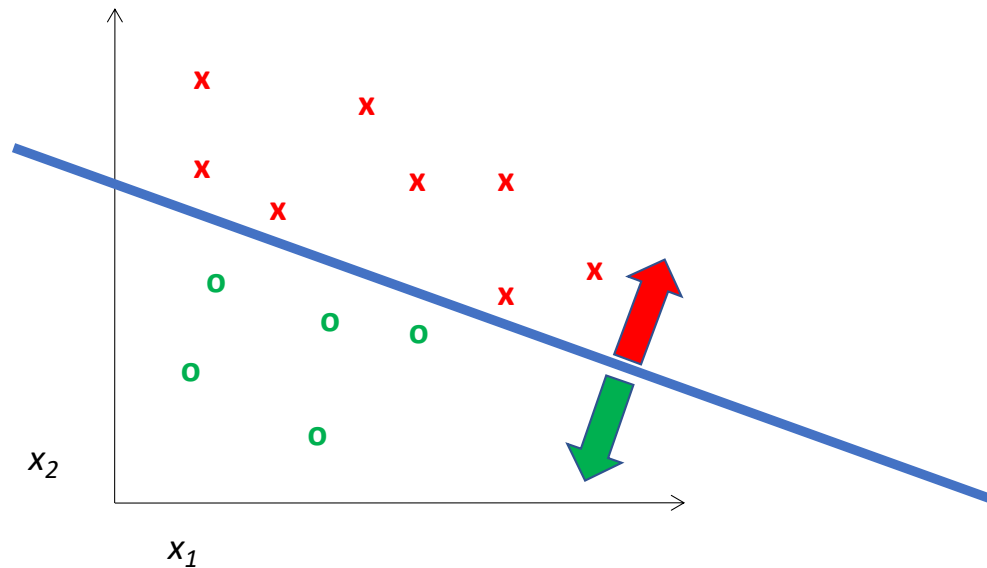
 ?

Other Classification Tasks



Learning a Classifier

- Given some set of features with corresponding labels, learn a function to predict the labels from the features.



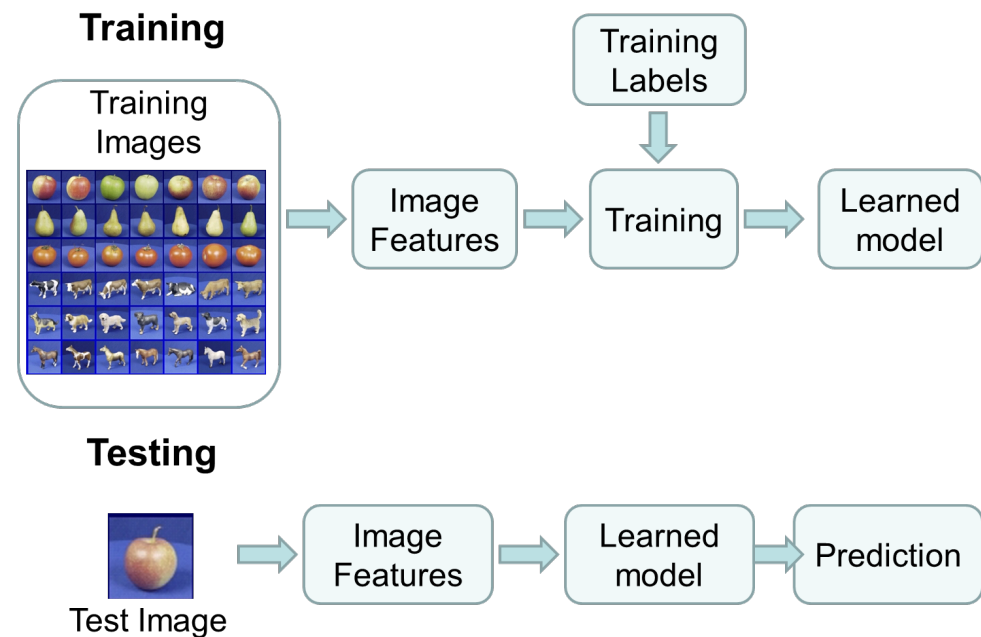
Learning a Classifier

- Apply a prediction function to the feature representation of the image to get the desired output:

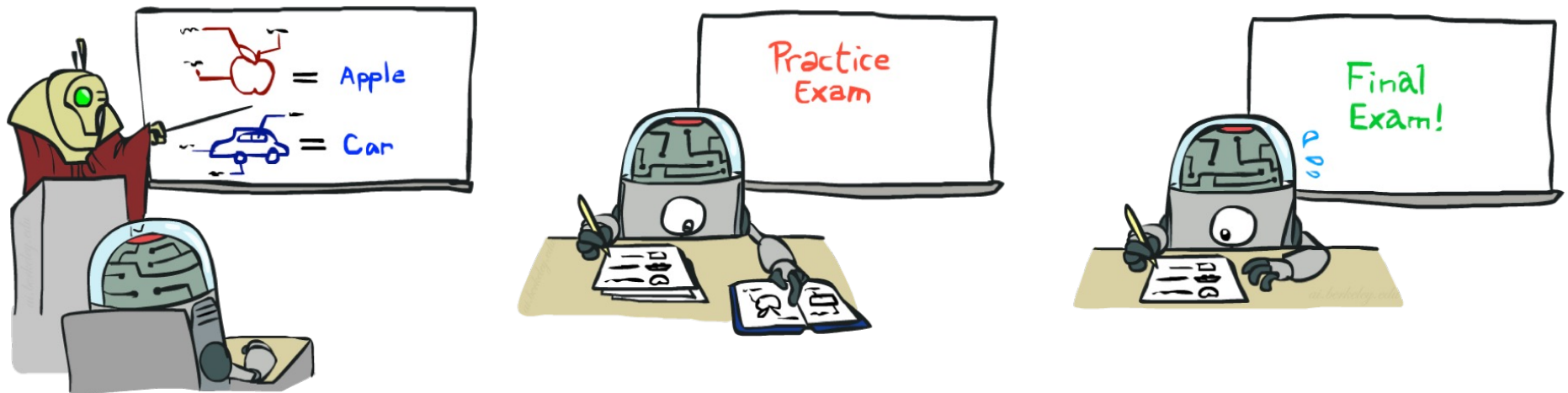
$f(\text{apple image}) = \text{"apple"}$

$f(\text{tomato image}) = \text{"tomato"}$

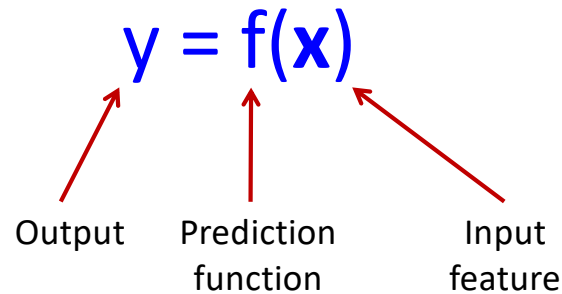
$f(\text{cow image}) = \text{"cow"}$



Training and Testing



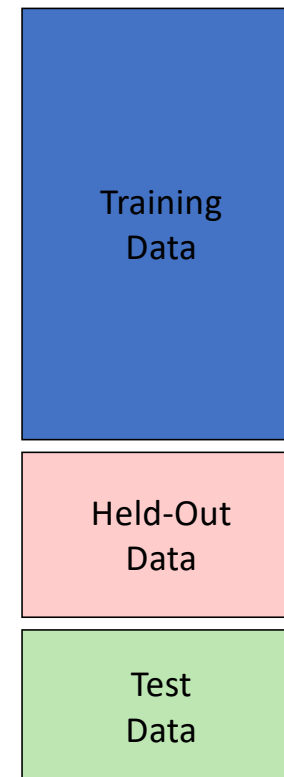
Training and Testing



- **Training:** given a *training set* of labeled examples $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$, estimate the prediction function f by minimizing the prediction error on the training set
- **Testing:** apply f to a never before seen *test example* $\mathbf{x}$ and output the predicted value $y = f(\mathbf{x})$
 - We also use *testing* as *inference* in many cases.

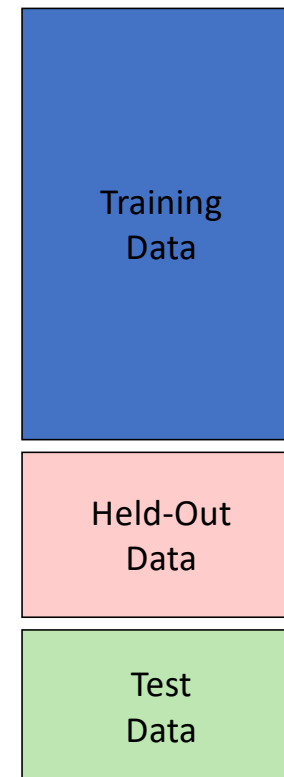
Training and Testing

- Data: labeled instances (e.g. emails marked spam/ham).
 - Training set;
 - Held out set (“development” or “validation” set);
 - Test set.
- Features: attribute-value pairs which characterize each instance.



Training and Testing

- Experimentation cycle:
 - Learn parameters (e.g. model probabilities) on training set;
 - (Tune hyperparameters on held-out set);
 - Compute accuracy of test set;
 - Very important: never “peek” at the test set!
- Evaluation
 - Many metrics possible;
 - Accuracy: fraction of instances predicted correctly.

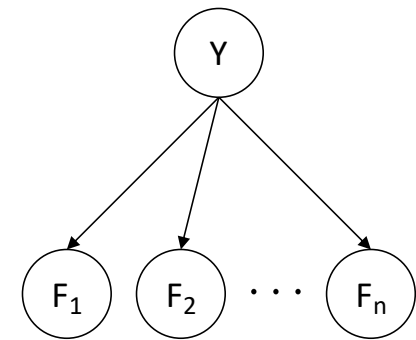


Example: Naïve Bayes for Digits

- Naïve Bayes: Assume all features are (conditionally) independent effects of the label.
- Simple digit recognition version:
 - One feature (variable) F_{ij} for each grid position $\langle i, j \rangle$.
 - Feature values are on / off, based on whether intensity is more or less than 0.5 in underlying image.
 - Each input maps to a feature vector, e.g.

 $\rightarrow \langle F_{0,0} = 0 \ F_{0,1} = 0 \ F_{0,2} = 1 \ F_{0,3} = 1 \ F_{0,4} = 0 \ \dots F_{15,15} = 0 \rangle$

- Naïve Bayes model: $P(Y|F_{0,0} \dots F_{15,15}) \propto P(Y) \prod_{i,j} P(F_{i,j}|Y)$



General Naïve Bayes

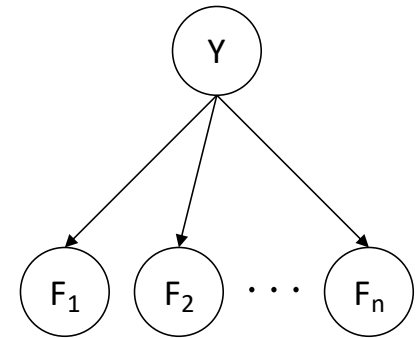
- A general **Naive Bayes** model:

$|Y|$ parameters

$$P(Y, F_1 \dots F_n) = P(Y) \prod_i P(F_i | Y)$$

$|Y| \times |F|^n$ values

$n \times |F| \times |Y|$ parameters



- We only have to specify how each feature depends on the class.
- Total number of parameters is **linear** in n .
- Model is very simplistic, but often works anyway.

General for Naïve Bayes

- Goal: compute posterior distribution over label variable Y .
 - Step 1: get joint probability of label and evidence for each label.

$$P(Y, f_1 \dots f_n) = \begin{bmatrix} P(y_1, f_1 \dots f_n) \\ P(y_2, f_1 \dots f_n) \\ \vdots \\ P(y_k, f_1 \dots f_n) \end{bmatrix} \xrightarrow{\text{blue arrow}} \begin{bmatrix} P(y_1) \prod_i P(f_i|y_1) \\ P(y_2) \prod_i P(f_i|y_2) \\ \vdots \\ P(y_k) \prod_i P(f_i|y_k) \end{bmatrix}$$

Step 2: sum to get probability of evidence.

Step 3: normalize by dividing Step 1 by Step 2.

$$\frac{\begin{bmatrix} P(y_1) \prod_i P(f_i|y_1) \\ P(y_2) \prod_i P(f_i|y_2) \\ \vdots \\ P(y_k) \prod_i P(f_i|y_k) \end{bmatrix}}{P(f_1 \dots f_n)} \xrightarrow{\text{blue arrow}} P(Y|f_1 \dots f_n)$$

Note: A curved arrow with a '+' sign points from the denominator $P(f_1 \dots f_n)$ to the division line in the fraction above.

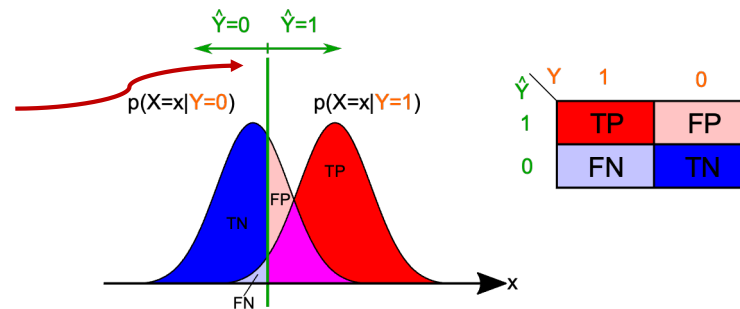
General Naïve Bayes

- What do we need in order to use Naïve Bayes?
 - Inference method (we just saw this part).
 - Start with a bunch of probabilities: $P(Y)$ and the $P(F_i|Y)$ tables;
 - Use standard inference to compute $P(Y|F_1...F_n)$.
 - Estimates of local conditional probability tables.
 - $P(Y)$, the prior over labels;
 - $P(F_i|Y)$ for each feature (evidence variable);
 - These probabilities are collectively called the *parameters* of the model and denoted by θ ;
 - Up until now, we assumed these appeared by magic, but...
 - ...they typically come from training data counts.

Classification Metrics

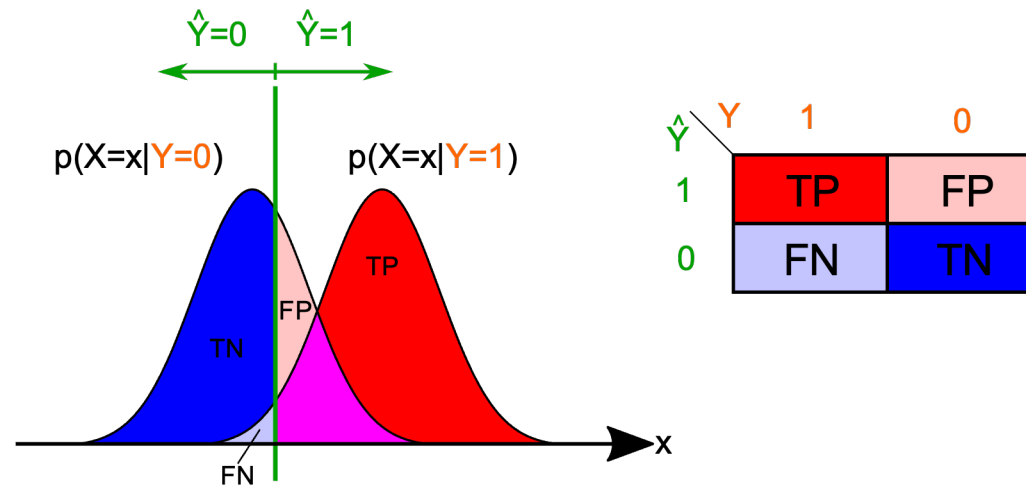
- Especially in binary classification, we define
 - **TP (True Positive)**: if the model predicts that a patient has a disease and the patient indeed has the disease, it is a true positive.
 - **TN (True Negative)**: if the model predicts that a patient does not have a disease and the patient indeed does not have the disease, it is a true negative.
 - **FP (False Positive, Type I error)**: if the model predicts that a patient has a disease but the patient does not have the disease, it is a false positive.
 - **FN (False Negative, Type II error)**: if the model predicts that a patient does not have a disease but the patient actually has the disease, it is a false negative.

Usually a thresholding is needed here!



Classification Metrics

- **Accuracy:** The ratio of correctly predicted instances to the total instances.
 - $(TP + TN) / (TP + FP + TN + FN)$
 - It measures the overall correctness of the model across all classes.
 - It is suitable when the classes are balanced.



Classification Metrics

- **Precision**: The ratio of correctly predicted positive instances to the total predicted positive instances.
 - $TP / (TP + FP)$
 - Indicates how many of the instances that the model labeled as positive are actually positive.
 - It is crucial when the cost of false positives is high.
- **Recall (Sensitivity, TPR)**: The ratio of correctly predicted positive instances to the total actual positive instances.
 - $TP / (TP + FN)$
 - Shows how many of the actual positive instances the model can identify correctly.
 - It is important when the cost of false negatives is high.

Classification Metrics

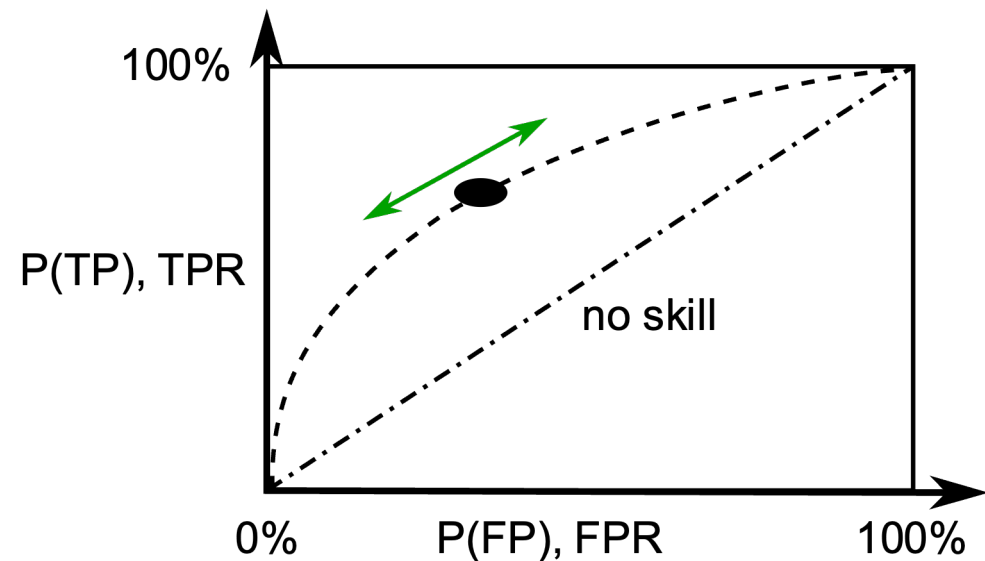
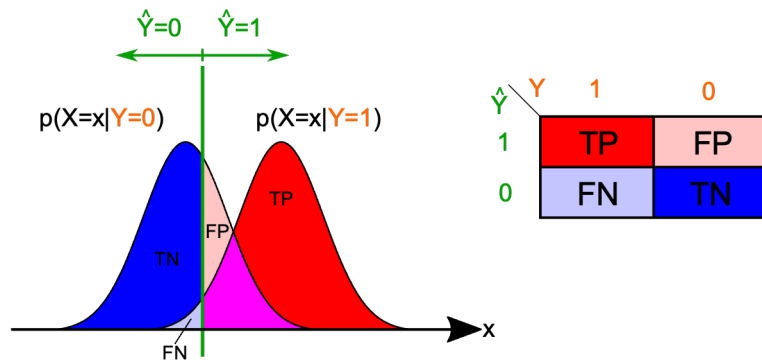
- **Specificity** (TNR): The ratio of correctly predicted negative instances to the total actual negative instances.
 - $TN / (TN + FP)$
 - Measures how many of the actual negative instances the model can identify correctly. It is crucial in scenarios where the cost of false positives is high.
 - **Precision**: Looks at the quality of positive predictions (how many of the predicted positives are actually positive).
 - **Specificity**: Looks at the quality of negative predictions (how many of the actual negatives are correctly identified as negative).
 - In summary, precision and specificity both address the issue of false positives but from different angles: precision focuses on the correctness of positive predictions, while specificity focuses on the ability to correctly identify negative instances.

Classification Metrics

- **F1-score**: The harmonic mean of precision and recall, providing a balance between both metrics.
 - $2 * \frac{Precision * Recall}{Precision + Recall}$
 - The F1 score is especially useful when you need to balance precision and recall.
 - It is a better metric than accuracy for imbalanced datasets.

ROC

- The **Receiver Operating Characteristic (ROC)** curve is a graphical representation for the trade-off between TPR (Sensitivity or Recall) and False Positive Rate (FPR) across different threshold settings.



AUC

- The Area Under the Curve (AUC) of a ROC curve is a single scalar value summarizing the overall performance of a binary classification model.
 - An AUC of 0.5 indicates no discriminative power (equivalent to random guessing), while an AUC of 1.0 indicates perfect classification.

